Parte 1: Generación de claves RSA

Antes de salir de fábrica, el coche genera su propio par de claves RSA para comunicarse de forma segura con la sede central de AutoDrive.

1. *Cálculo manual: Supongamos una configuración RSA simplificada donde el sistema elige dos números primos pequeños: p = 11 y q = 13*
   a. *Genere las claves pública y privada.*

• Genera claves publicas y privadas

$p = 11$

$q = 13$

- Calcular n

$n = p \cdot q = 11 \cdot 13 = 143$

- Calcular $\varphi(n) = (p-1)(q-1)$

$\varphi(143) = (11-1)(13-1)$

$\varphi(143) = 120$

- Elegir un numero entre $1 < e < 120$

$\gcd(e, 120) = 1$

$e = 7$

$\gcd(7, 120) = 1$

$120 = 7 \times 17 + 1$

$7 = 1 \times 7 + 0$

- Calcular clave privada (d)

$(e \cdot d) \bmod \varphi(n) = 1$

$(7 \cdot d) \bmod 120 = 1$

$d \cdot 7 \equiv 1 \pmod{120}$

$1 = 120(1) - 17(7)$

$-17 \cdot 7 \equiv 1 \pmod{120}$

$d = -17 + 120 = 103$

Claves finales

Clave publica:

$(e, n) = (7, 143)$

Clave privada:

$(d, n) = (103, 143)$

   b. *Verifique que se puedan usar para cifrar y descifrar.*

• Verificación (cifrar y descifrar)

- Cifrado mensaje = 9 $(e, n) = (7, 143)$

$C = m^e \bmod n$

$C = 9^7 \bmod 143$

$C = 48$

- Descifrado mensaje cifrado = 48 $(d, n) = (103, 143)$

$m = c^d \bmod n$

$m = 48^{103} \bmod 143$

$m = 9$

2. *Lógica de implementación: ¿Por qué es computacionalmente imposible para un hacker encontrar "d" si solo conoce "n" y "e" en un sistema real de 2048 bits?*

En un sistema RSA real, resulta computacionalmente inviable que un atacante pueda obtener la clave privada d conociendo únicamente la clave pública, es decir, los valores n y e. Esto se debe a que el cálculo de d depende directamente del valor de la función totiente de Euler $\varphi(n)$, la cual se obtiene a partir de los factores primos de n, es decir, p y q.

En la práctica, el valor n se genera como el producto de dos números primos extremadamente grandes, cada uno con cientos de dígitos. Para poder calcular $\varphi(n)$, un atacante tendría que factorizar n y recuperar esos dos números primos. Sin embargo, el problema de la factorización de enteros grandes no tiene, hasta la fecha, un algoritmo eficiente que pueda resolverlo en un tiempo razonable cuando los números son de gran tamaño, como ocurre en RSA de 2048 bits. (Menezes, 1997)

Un número de 2048 bits tiene aproximadamente 617 dígitos decimales. Para una idea de escala:

- El número de átomos en el universo observable es de unos 10^{80}.
- Un número de 2048 bits llega hasta 10^{616}.

Si tomamos la supercomputadora más rápida del mundo (como *Frontier* en EE.UU., que opera a ~1.2 exaflops), y la dedicamos exclusivamente a esta tarea.

$$1\ exaflplop\ \approx 10^{18}\ operaciones\ por\ segundo$$

$$2^{112}\ \approx 5 \times 10^{33}\ operaciones\ totales$$

$$Tiempo\ = 5 \times 10^{33} / 10^{18}\ \approx 5 \times 10^{15}\ segundos$$

$$5 \times 10^{15} segundos\ \approx 158{,}000{,}000\ años$$

En conclusión, aunque la clave pública (n, e) es conocida y puede ser distribuida libremente, la clave privada $d$permanece segura porque obtenerla implicaría resolver un problema computacionalmente inabordable con la tecnología actual. Esto garantiza la confidencialidad y autenticidad de las comunicaciones en sistemas como el de AutoDrive.

Fuentes:

- Menezes, A. J. (1997). *Handbook of Applied Cryptography*. CRC Press.